

فاز ۵: ساختمان داده و طراحی الگوریتم – کامل و عمیق

عمیق

هدف این بخش حفظ کردن کد نیست، بلکه درک نحوه کار و توانایی انتخاب ابزار مناسب برای هر مسئله است. در مصاحبه، ازت می‌خواند که مسئله رو روی تخته حل کنی و بعد تحلیلش کنی که چرا این راه حل خوبه.

5.1 تحلیل پیچیدگی (Big O Notation)

سوال: Big O چیست و چرا اهمیت دارد؟ (این سوال O همیشه می‌پرسن)

جواب: Big O یک نمادگذاری ریاضی است که نرخ رشد (حد بالا) زمان اجرا یا فضای مصرفی یک الگوریتم را بر اساس اندازه ورودی (n) توصیف می‌کند.

هدفش چیست؟ مقایسه مقیاس‌پذیری (Scalability) الگوریتم‌ها. وقتی n به سمت بی‌نهایت میل می‌کند، کدام الگوریتم بهتر عمل می‌کند؟ (ثابت‌ها را فاکتور می‌گیریم).

! سوال مصاحبه‌ای ۱: همه پیچیدگی‌های زمانی معروف را به ترتیب از بهترین به بدترین نام ببر و برای هر کدام یک مثال واقعی در جاوا بزن

نماد	نام	مثال واقعی (جاوا / دنیای واقعی)	توضیح
$O(1)$	ثابت (Constant)	دسترسی به یه خانه آرایه <code>arr[5]</code> ، درج در <code>HashMap</code>	مستقل از n ، همیشه زمان ثابت است. بهترین.
$O(\log n)$	لگاریتمی (Logarithmic)	جستجوی دودویی (<code>binarySearch</code>)، جستجو در <code>TreeMap</code>	با افزایش n ، زمان خیلی کم زیاد می‌شود. عالی.
$O(n)$	خطی (Linear)	یک حلقه <code>for</code> ساده، جستجوی خطی در	زمان به نسبت مستقیم با n زیاد

نماد	نام	مثال واقعی (جاوا / دنیای واقعی)	توضیح
		ArrayList	می‌شود. خوب.
$O(n \log n)$	خطی-لگاریتمی (Linearithmic)	مرتب‌سازی سریع (QuickSort)، مرتب‌سازی ادغامی (MergeSort)	بهترین پیچیدگی برای الگوریتم‌های مرتب‌سازی عمومی.
$O(n^2)$	مربعی (Quadratic)	دو حلقه for تو در تو، مرتب‌سازی حبابی (BubbleSort)	بد. با بزرگ شدن n خیلی زود کند می‌شود.
$O(2^n)$	نمایی (Exponential)	محاسبه بازگشتی فیبوناچی بدون memoization	فاجعه. فقط برای n های خیلی کوچک قابل استفاده است.

چطور محاسبه کنیم؟

• حلقه for ساده: $O(n)$

• حلقه‌های تو در تو: تو n (مثلاً ۲ حلقه) $= O(n^2)$

• نصف کردن متوالی فضای جستجو: $O(\log n)$

• عملیات پشت سر هم: پیچیدگی بزرگترین عملیات غالب می‌شود. $(O(n) + O(\log n) = O(n))$

5.2 ساختمان داده‌های اصلی (Core Data Structures)

مصاحبه‌کننده می‌خواهد بداند تو فقط ArrayList و HashMap بلد نیستی، بلکه می‌دانی کی از کدام استفاده کنی.

۱. آرایه (Array)

• چیست؟ بلوکی از خانه‌های پشت سر هم در حافظه.

• مزیت: دسترسی تصادفی (get) با $O(1)$.

• عیب: سایز ثابت. درج/حذف از وسط با $O(n)$.

• کاربرد جاوا: `String`, `int[]`.

۲. لیست پیوندی (Linked List)

- چیست؟ زنجیره‌ای از گره‌ها. هر گره داده و یک اشاره‌گر به گره بعدی (و قبلی در حالت دوطرفه) دارد.
- مزیت: درج/حذف از ابتدا/انتها با $O(1)$. سائز پویا.
- عیب: دسترسی تصادفی (get) با $O(n)$ چون باید از اول لیست حرکت کرد.
- کاربرد جاوا: LinkedList (حافظه بیشتر نسبت به ArrayList).

۳. پشته (Stack)

- چیست؟ ساختمان داده LIFO (Last-In, First-Out). آخرین چیزی که می‌ذاری، اولین چیزی که برمی‌داری. مثل پشته کتاب.
- عملیات اصلی:
- push(): گذاشتن روی پشته ($O(1)$).
- pop(): برداشتن از روی پشته ($O(1)$).
- peek(): دیدن بالاترین عنصر بدون برداشتن ($O(1)$).
- کاربرد: دکمه Back در مرورگر، Call Stack در JVM، ارزیابی عبارات ریاضی.
- جاوا: کلاس Stack قدیمی است. به جایش Deque (مثلاً ArrayDeque) توصیه می‌شود.

۴. صف (Queue)

- چیست؟ ساختمان داده FIFO (First-In, First-Out). مثل صف نانوايي. اولین کسی که می‌آید، اولین نفر خارج می‌شود.
- عملیات اصلی:
- enqueue()/offer(): اضافه به انتها ($O(1)$).
- dequeue()/poll(): برداشتن از ابتدا ($O(1)$).

• کاربرد: مدیریت درخواست‌ها در سرور، صف چاپگر، Message Brokers (مثل RabbitMQ).

• جاوا: Queue یک Interface است. پیاده‌سازی‌ها: LinkedList, PriorityQueue.

۵. هاش مپ (Hash Map) / جدول درهم‌سازی (Hash Table)

• چیست؟ ذخیره‌سازی زوج‌های کلید-مقدار. با استفاده از یک تابع هاش (hashCode())، کلید به یک ایندکس در یک آرایه (Bucket) تبدیل می‌شود.

• مزیت: میانگین $O(1)$ برای get, put, delete.

• عیب: بدترین حالت $O(n)$ در صورت برخورد (Collision) زیاد. بینظم.

• جزئیات حیاتی (در جاوا):

• برخورد (Collision): وقتی دو کلید، hashCode یکسان داشته باشند. راه حل: زنجیره‌سازی (Chaining).

هر سطل آرایه، یک لیست پیوندی است. در Java 8، اگر لیست طولانی شود (> 8)، برای بهبود به $O(\log n)$ به درخت سرخ-سیاه تبدیل می‌شود.

• اهمیت equals و hashCode: تابع هاش می‌گوید در کدام سطل بگردد. equals می‌گوید کدام کلید در آن سطل دقیقاً همانی است که می‌خواهیم.

5.3 الگوریتم‌های پایه

۱. جستجو (Searching)

• جستجوی خطی (Linear Search): تک‌تک خونه‌ها را پیمایش کن. $O(n)$. فقط برای داده‌های نامرتب.

• جستجوی دودویی (Binary Search): $O(\log n)$. عالی است، اما فقط روی داده‌های مرتب کار می‌کند. وسط چک می‌کند. اگر هدف بزرگتر بود، نیمه چپ را دور بریز. اگر کوچکتر بود، نیمه راست را دور بریز.

۲. مرتب‌سازی (Sorting)

• مرتب‌سازی حبابی (Bubble Sort): $O(n^2)$. فقط آموزشی!

• مرتب‌سازی سریع (Quick Sort): متوسط $O(n \log n)$. از یک عنصر محوری (pivot) استفاده می‌کند. معمولاً در عمل سریع‌ترین است. (الگوریتم Arrays.sort () جاوا برای primitives).

• مرتب‌سازی ادغامی (Merge Sort): همیشه $O(n \log n)$. پایدار (Stable) است. از روش تقسیم و حل (Divide & Conquer) استفاده می‌کند. نیاز به حافظه اضافی دارد. (الگوریتم Collections.sort () و Arrays.sort () برای اشیاء).

5.4 رویکردهای طراحی الگوریتم

! سوال مصاحبه‌ای ۲: مسائل را با چه رویکردهایی می‌توان حل کرد؟

۱. تقسیم و حل (Divide and Conquer):

• ایده: مسئله بزرگ را به زیرمسئله‌های کوچکتر و مشابه بشکن. زیرمسئله‌ها را حل کن. نتایج را ترکیب کن.

• مثال: Merge Sort، Quick Sort، جستجوی دودویی.

۲. برنامه‌نویسی پویا (Dynamic Programming - DP):

• ایده: وقتی زیرمسئله‌ها همپوشانی (Overlap) دارند (مثلاً تابع فیبوناچی، fib(5) و fib(4) هر دو به fib(3) نیاز دارند). به جای محاسبه مجدد، نتایج زیرمسئله‌ها را در یک حافظه (Memo) ذخیره کن.

• مثال کلاسیک: محاسبه فیبوناچی. از $O(2^n)$ به $O(n)$ کاهش می‌یابد.

۳. حریصانه (Greedy):

• ایده: در هر قدم، بهترین انتخاب محلی را انجام بده، به این امید که به بهترین جواب کلی برسی. همیشه جواب بهینه کلی را نمی‌دهد.

• مثال: الگوریتم دایکسترا (کوتاه‌ترین مسیر)، مسئله بازگرداندن پول خرد با کمترین تعداد سکه.

۴. پسگرد (Backtracking):

• ایده: مانند یک هزارتو، تمام مسیرهای ممکن را امتحان کن و هر جا به بن‌بست رسیدی، به عقب برگرد و مسیر دیگری را امتحان کن.

• مثال: مسئله هشت وزیر (Eight Queens)، حل جدول سودوکو.

5.5 جمع‌بندی مصاحبه‌ای: مقایسه‌های کلیدی

! سوال مصاحبه‌ای ۳: ArrayList در مقابل LinkedList؟

• ArrayList: دسترسی تصادفی $O(1)$ ، درج در وسط $O(n)$.

• LinkedList: دسترسی تصادفی $O(n)$ ، درج در وسط $O(1)$ (بعد از پیدا کردن مکان).

• قانون: در ۹۰٪ موارد ArrayList به دلیل بهینگی برای حافظه کش CPU و کارایی عمومی بهتر است.

! سوال مصاحبه‌ای ۴: HashMap در مقابل TreeMap؟

• HashMap: میانگین $O(1)$ برای عملیات اصلی. بدون ترتیب.

• TreeMap: $O(\log n)$ برای عملیات اصلی. کلیدها را به ترتیب (مرتب‌شده) نگه می‌دارد.